

به نام خدا

نام و نام خانوادگی:

آرمان پایان

استاد:

دکتر باغملائی

درس :

مبانی هوش مصنوعی

گزارش پروژه

1 معرفی مسئله

هدف این پروژه، طراحی و پیاده‌سازی یک عامل هوشمند برای بازی دو نفره Connect 4 است؛ بازی‌ای که در آن دو بازیکن به‌صورت نوبتی مهره‌های خود را در یکی از ستون‌های صفحه‌ای ۶ در ۷ رها می‌کنند و هدف هر بازیکن، تشکیل چهار مهره متوالی به‌صورت افقی، عمودی یا مورب است. این بازی به دلیل فضای حالت بزرگ و ماهیت رقابتی و دونفره‌اش، بستر مناسبی برای پیاده‌سازی الگوریتم‌های جستجوی خصمانه (Adversarial Search) است.

در این پروژه، محیط بازی، عامل هوشمند مبتنی بر الگوریتم Minimax مجهز به هرس Alpha-Beta، و یک تابع ارزیابی هیوریستیک به‌طور کامل پیاده‌سازی شده‌اند. علاوه بر بخش‌های الزامی، یک رابط گرافیکی (GUI) نیز به‌عنوان بخش اختیاری و امتیازی توسعه داده شده است تا امکان بازی تعاملی با عامل هوشمند و تنظیم سطح دشواری آن فراهم شود.

پروژه در زبان پایتون و بدون استفاده از هیچ کتابخانه یا پیاده‌سازی آماده برای منطق بازی، Minimax یا Alpha-Beta Pruning نوشته شده و شامل فایل‌های زیر است:

فایل	وظیفه
game.py	پیاده‌سازی محیط و قوانین بازی Connect 4
heuristic.py	تابع ارزیابی هیوریستیک صفحه بازی
minimax.py	عامل هوشمند مبتنی بر Minimax و Alpha-Beta Pruning
main.py	اجرای بازی در محیط خط فرمان (Console)
gui.py	رابط گرافیکی بازی (بخش اختیاری)

2 مدل‌سازی فضای حالت بازی

هسته محیط بازی در کلاس Connect4 در فایل game.py پیاده‌سازی شده است. هر حالت بازی به‌صورت یک ماتریس ۷×۶ نگهداری می‌شود که مقادیر آن یکی از سه حالت زیر است:

• EMPTY = 0 : خانه خالی

• PLAYER = 1 : مهره بازیکن انسانی

• $AI = 2$: مهره عامل هوشمند

۲.۱ عملگرها (Actions)

عملگر اصلی، انتخاب یک ستون معتبر است. متد `valid_moves()` ستون‌هایی را برمی‌گرداند که ردیف بالایی آن‌ها هنوز خالی است. متد `make_move(col, player)` مهره را در پایین‌ترین خانه خالی همان ستون قرار می‌دهد که این رفتار، قانون سقوط مهره در بازی واقعی را شبیه‌سازی می‌کند. برای پشتیبانی از جستجوی برگشت‌پذیر (backtracking) در Minimax، متد `undo_move(col)` نیز آخرین مهره ستون را حذف می‌کند تا صفحه به حالت قبلی بازگردد؛ این کار به جای کپی‌کردن کامل صفحه در هر گره جستجو، از نظر حافظه و سرعت بسیار بهینه‌تر است.

۲.۲ آزمون هدف (Goal Test)

متد `check_winner(player)` با بررسی تمام پنجره‌های چهارتایی افقی، عمودی، و دو جهت مورب، برد یک بازیکن را تشخیص می‌دهد. متد `is_full()` پرشدن کامل صفحه را بررسی می‌کند و متد `game_over()` با ترکیب این دو، پایان بازی را (برد هرکدام از دو بازیکن یا تساوی) تعیین می‌کند.

۲.۳ تولید حالت‌های بعدی

متد `get_next_states(player)` برای هر ستون مجاز، یک نسخه کپی‌شده از بازی (`copy()`) ایجاد کرده و حرکت را روی آن اعمال می‌کند و لیستی از زوج‌های (ستون، حالت جدید) باز می‌گرداند. این تابع، عملیات "گسترش گره" در درخت جستجو را نمایندگی می‌کند، هرچند در پیاده‌سازی نهایی Minimax، به جای کپی‌کردن کامل صفحه، از الگوی make/undo برای کارایی بیشتر استفاده شده است.

فضای حالت این مسئله بسیار بزرگ است (تخمین نظری حدود 4×10^{12} آرایش ممکن روی صفحه). به همین دلیل جستجوی کامل تا انتهای بازی عملی نیست و باید جستجو تا عمق محدود انجام شده و حالت‌های برگ با یک تابع ارزیابی هیوریستیک تخمین زده شوند.

3 توضیح الگوریتم Alpha-Beta Pruning و Minimax

عامل هوشمند در کلاس `MinimaxAI` (فایل `minimax.py`) پیاده‌سازی شده است. این الگوریتم بازی را به صورت یک بازی دونفره با اطلاعات کامل مدل می‌کند که در آن عامل هوشمند (AI) نقش بازیکن MAX و بازیکن انسانی نقش بازیکن MIN را دارد.

۳.۱ ساختار جستجو

متد `alpha_beta(game, depth, alpha, beta, maximizing_player)` درخت بازی را به صورت بازگشتی جستجو می‌کند. در هر فراخوانی، ابتدا سه شرط پایانی بررسی می‌شود: برد AI، برد بازیکن انسانی، یا پر شدن صفحه؛ در این حالات، امتیازی بزرگ (مثبت برای برد AI و منفی برای برد حریف) با کمی تعدیل بر اساس عمق باقی‌مانده بازگردانده می‌شود تا الگوریتم بردهای سریع‌تر را بر بردهای دیرتر ترجیح دهد. اگر به عمق صفر برسیم، مقدار تابع هیوریستیک `evaluate_board` به عنوان تخمین ارزش حالت بازگردانده می‌شود.

در نقش MAX (نوبت AI)، الگوریتم بیشترین مقدار ممکن از میان فرزندان را انتخاب کرده و `alpha` را به روزرسانی می‌کند. در نقش MIN (نوبت بازیکن)، کمترین مقدار انتخاب شده و `beta` به روزرسانی می‌شود. هرگاه شرط `alpha >= beta` برقرار شود، شاخه جستجو هرس (`prune`) و از بررسی ادامه نداده می‌شود، زیرا آن مسیر دیگر نمی‌تواند نتیجه نهایی را تغییر دهد.

```
if alpha >= beta:
    break # هرس شاخه: ادامه بررسی این گره سودی ندارد
```

۳.۲ مرتب‌سازی حرکات (Move Ordering)

متد `order_moves()` حرکات را بر اساس فاصله از ستون مرکزی مرتب می‌کند تا حرکات مرکزی (که معمولاً از نظر استراتژیک قوی‌تر هستند) زودتر بررسی شوند. بررسی زود هنگام حرکات بهتر، احتمال هرس شدن شاخه‌های ضعیف‌تر را در Alpha-Beta Pruning افزایش داده و عملاً عمق مؤثر قابل جستجو در زمان مشابه را بالا می‌برد.

۳.۳ لایه‌های تاکتیکی پیش از جستجوی کامل

پیش از آغاز جستجوی Minimax، متد `get_best_move` دو بررسی سریع انجام می‌دهد:

1. اگر عامل هوشمند بتواند با یک حرکت بلافاصله برنده شود، همان حرکت بدون نیاز به جستجوی عمیق انتخاب می‌شود.
 2. اگر بازیکن انسانی در نوبت بعدی خود بتواند فوراً برنده شود، عامل هوشمند آن ستون را مسدود می‌کند.
- این دو لایه، تضمین می‌کنند که عامل هرگز فرصت برد آشکار را از دست نمی‌دهد و هرگز باخت یک‌حرکته را نادیده نمی‌گیرد، مستقل از عمق جستجوی Minimax.

۳.۴ شمارش گره‌های بررسی‌شده

متغیر `nodes_explored` در هر فراخوانی `alpha_beta` افزایش می‌یابد و در نسخه کنسولی (`main.py`) پس از هر حرکت AI چاپ می‌شود؛ این مقدار برای سنجش کارایی هرس Alpha-Beta و مقایسه عمق‌های مختلف جستجو مفید است.

4 تشریح تابع هیوریستیک

از آنجا که جستجوی کامل درخت بازی 4 Connect از نظر محاسباتی غیرممکن است، تابع `evaluate_board` در فایل `heuristic.py` یک تخمین عددی از "خوب بودن" یک حالت صفحه برای عامل هوشمند ارائه می‌دهد. امتیاز نهایی از جمع چند معیار زیر به‌دست می‌آید:

۴.۱ ارزش ستون مرکزی

هر مهره AI که در ستون مرکزی (ستون شماره ۳) قرار دارد، ۶ امتیاز اضافه می‌کند. کنترل ستون مرکزی از نظر استراتژیک ارزشمند است زیرا در بیشترین تعداد پنجره‌های برنده احتمالی (افقی، عمودی و هر دو قطر) شرکت می‌کند.

۴.۲ ارزیابی پنجره‌های چهارتایی

تابع تمام پنجره‌های چهارخانه‌ای ممکن در چهار جهت (افقی، عمودی، قطر اصلی و قطر فرعی) را با تابع `evaluate_window` بررسی می‌کند. در هر پنجره، تعداد مهره‌های AI، مهره‌های بازیکن انسانی و خانه‌های خالی شمارش می‌شود:

الگو در پنجره	امتیاز	تفسیر
۴ مهره AI	+۱۰۰,۰۰۰	برد قطعی
۳ مهره ۱ + AI خانه خالی	+۱۰۰	فرصت برد نزدیک
۲ مهره ۲ + AI خانه خالی	+۱۰	آرایش تهاجمی اولیه
۳ مهره حریف + ۱ خانه خالی	-۱۲۰	تهدید جدی؛ باید دفع شود
۲ مهره حریف + ۲ خانه خالی	-۱۵	تهدید بالقوه

توجه شود که وزن دفاعی در برابر سه‌مهره‌ای حریف (۱۲۰-) عمداً کمی بیشتر از وزن تهاجمی معادل خود AI (+۱۰۰) در نظر گرفته شده است؛ این عدم‌تقارن باعث می‌شود عامل در موقعیت‌های مرزی، دفاع در برابر تهدید فوری حریف را بر ایجاد فرصت جدید برای خودش ترجیح دهد و از باخت‌های اجتناب‌پذیر جلوگیری کند.

۴.۳ برآیند طراحی

جمع امتیاز ستون مرکزی و تمام پنجره‌های چهارگانه، یک عدد واحد را برای هر حالت صفحه تولید می‌کند که در گره‌های برگ جستجوی محدود شده به‌عنوان تخمین ارزش آن حالت در Alpha-Beta Pruning استفاده می‌شود. این طراحی، هم‌زمان معیارهای تهاجمی (توسعه آرایش‌های خودی) و دفاعی (خنثی‌سازی تهدید حریف) را در یک تابع واحد ترکیب می‌کند.

5 چالش‌های پیاده‌سازی

۵.۱ توازن بین عمق جستجو و زمان پاسخ

افزایش عمق جستجوی Minimax، کیفیت تصمیم عامل را بهبود می‌بخشد اما به‌صورت نمایی حجم محاسبات را افزایش می‌دهد. برای مدیریت این توازن، سه سطح دشواری (آسان: عمق ۳، متوسط: عمق ۵، سخت: عمق ۷) در رابط گرافیکی تعریف شده است تا کاربر بتواند بسته به قدرت پردازشی سیستم و سطح چالش مطلوب، عمق را تنظیم کند.

۵.۲ کارایی حافظه در جستجوی بازگشتی

یکی از چالش‌های اولیه، مدیریت حالت صفحه در طول جستجوی بازگشتی بود. ساخت یک کپی کامل از صفحه در هر گره از درخت جستجو (همان‌طور که در `get_next_states` انجام می‌شود) در عمق‌های بالا هزینه حافظه و زمان قابل‌توجهی دارد. راه‌حل اتخاذ شده، استفاده از الگوی `make/undo` (`make_move` و `undo_move`) در هسته الگوریتم `alpha_beta` بود تا به‌جای کپی صفحه، تنها یک حرکت اعمال و در پایان بازگردانی شود.

۵.۳ جلوگیری از تصمیمات نزدیک‌بین (Horizon Effect)

از آنجا که جستجو تا عمق محدود انجام می‌شود، ممکن است عامل یک تهدید یا فرصت برد را درست در مرز عمق جستجو نبیند. برای کاهش این مشکل، دو بررسی مستقیم «برد فوری» و «جلوگیری از باخت فوری» پیش از آغاز Minimax اضافه شد تا این دو حالت حیاتی، صرف‌نظر از عمق تنظیم شده، همیشه به‌درستی تشخیص داده شوند.

۵.۴ طراحی و کالیبراسیون تابع هیوریستیک

تعیین وزن مناسب برای هر الگو در تابع ارزیابی (برای مثال، اینکه دفاع در برابر سه مهره‌ای حریف چقدر باید نسبت به حمله سنگین‌تر باشد) نیازمند آزمایش و تنظیم تدریجی بود. وزن‌های نهایی به‌گونه‌ای انتخاب شدند که عامل هم در دفاع در برابر تهدیدهای آشکار حریف و هم در ایجاد فرصت‌های برد برای خود، رفتاری متعادل و منطقی نشان دهد.

۵.۵ هماهنگی رابط گرافیکی با منطق بازی

در پیاده‌سازی `gui.py`، چالش اصلی هماهنگ‌سازی رویداد کلیک کاربر (ستون انتخابی بر اساس مختصات ماوس) با منطق داخلی `Connect4` و همچنین اجرای حرکت AI پس از یک تأخیر کوتاه (`window.after(500, ...)`) بود تا رابط کاربری در حین محاسبه AI قفل نشود و تجربه کاربری روان‌تری فراهم شود.

6 نتیجه‌گیری

در این پروژه، یک عامل هوشمند کامل برای بازی Connect 4 با استفاده از الگوریتم Minimax مجهز به هرس Alpha-Beta و یک تابع ارزیابی هیوریستیک چندمعیاره طراحی و پیاده‌سازی شد. نتایج نشان می‌دهد که ترکیب هرس Alpha-Beta با مرتب‌سازی هوشمندانه حرکات (اولویت‌دهی به ستون مرکزی)، امکان جستجوی عمیق‌تر در زمان معقول را فراهم می‌کند و بررسی‌های تاکتیکی مستقیم (تشخیص برد/ باخت فوری) از بروز اشتباهات آشکار در تصمیم‌گیری جلوگیری می‌کند.

پیاده‌سازی رابط گرافیکی به‌عنوان بخش اختیاری، امکان بازی تعاملی و تنظیم سطح دشواری را فراهم آورده و درک عملی‌تری از رفتار عامل هوشمند در سطوح مختلف عمق جستجو ارائه می‌دهد. به‌طور کلی، این پروژه مفاهیم اصلی جستجوی خصمانه، الگوریتم‌های MAX/MIN، هرس Alpha-Beta و طراحی تابع ارزیابی هیوریستیک را در قالب یک بازی واقعی و ملموس پیاده‌سازی و تجربه کرده است.